



Universidad Técnica Estatal de Quevedo

Tema:

Informe de Práctica en Clase

Autor:

Ernesto Gregory Luna Mora

Cristhian Daniel Pacheco Cárdenas

Robinson Rodrigo Cando Moreno

Profesor:

Gleiston Ciceron Guerrero Ulloa

Materia:

Aplicaciones Distribuidas

Quevedo-Ecuador

2026

Introducción

Hoy en día muchos programas se comunican entre sí aunque se encuentren en computadoras diferentes. Para que eso sea posible, se utilizan mecanismos llamados sockets, los cuales funcionan como un punto de conexión entre dos programas que quieren intercambiar información a través de la red.

Objetivo

El objetivo de esta práctica fue implementar un sistema de mensajería usando dos tipos de comunicación: TCP y UDP con sus clientes y servidores respectivos

Marco Teórico

Un socket es un punto de comunicación en red mediante el cual un proceso puede enviar y recibir datos. Este queda definido por una dirección IP, un puerto y un protocolo específico (UDP o TCP) [1]. La comunicación en la red ocurre entre ellos, no directamente entre procesos, y su comportamiento depende del protocolo asociado al socket.

Herramientas e Implementación

Herramientas utilizadas:

- Java 21 (LTS)
- Maven Archetype
- IntelliJ IDEA

El proyecto se organizó con la estructura estándar de Maven, separando las clases TCP y UDP en paquetes distintos.

Sockets TCP

Se implementaron dos clases: un servidor que escucha en el puerto 9000 y puede atender varios clientes al mismo tiempo, donde cada uno recibe su propio hilo de atención. El servidor reconoce tres comandos: HORA, ECO y SALIR. Por otro lado, el cliente se conecta al servidor e interactúa con él desde la consola, enviando comandos y mostrando las respuestas.

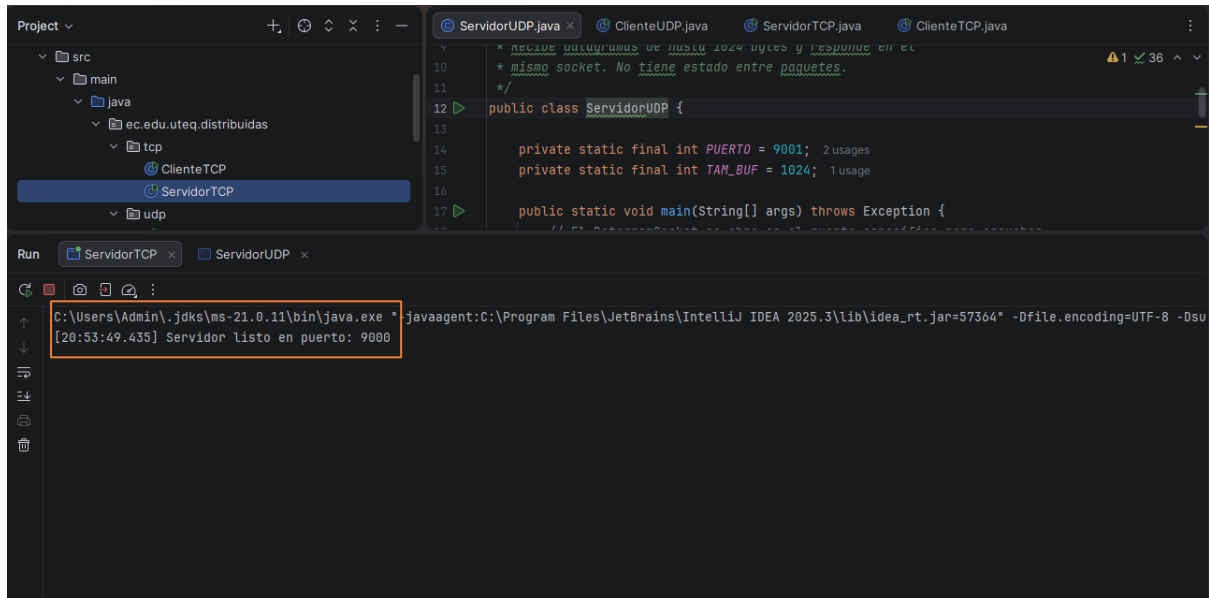


Ilustración 1 ServidorTCP

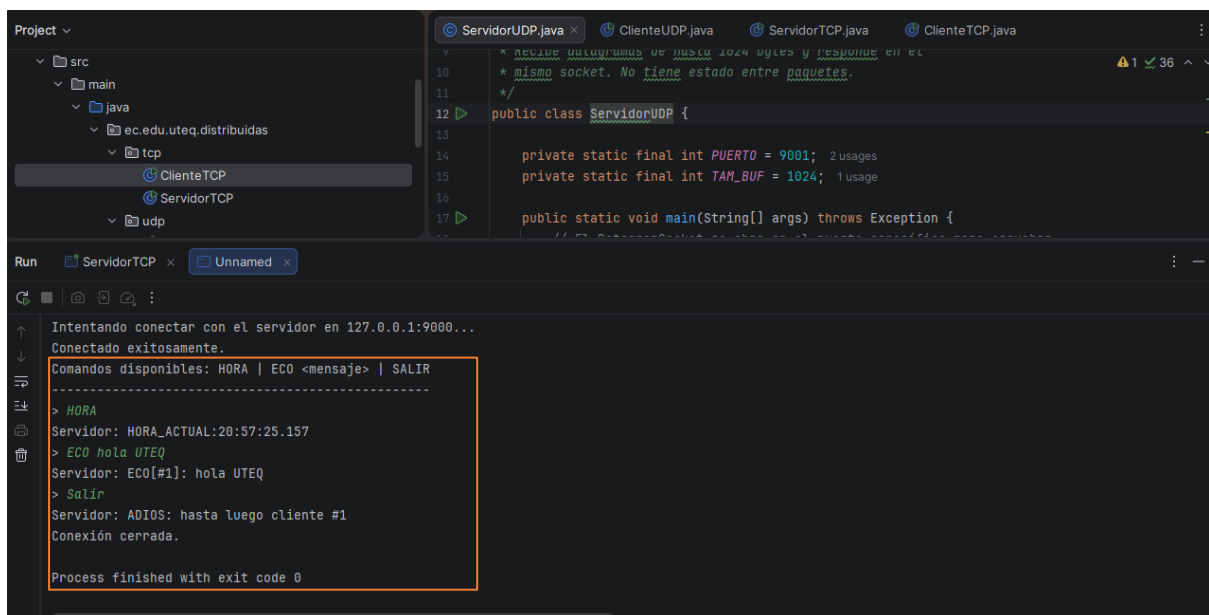
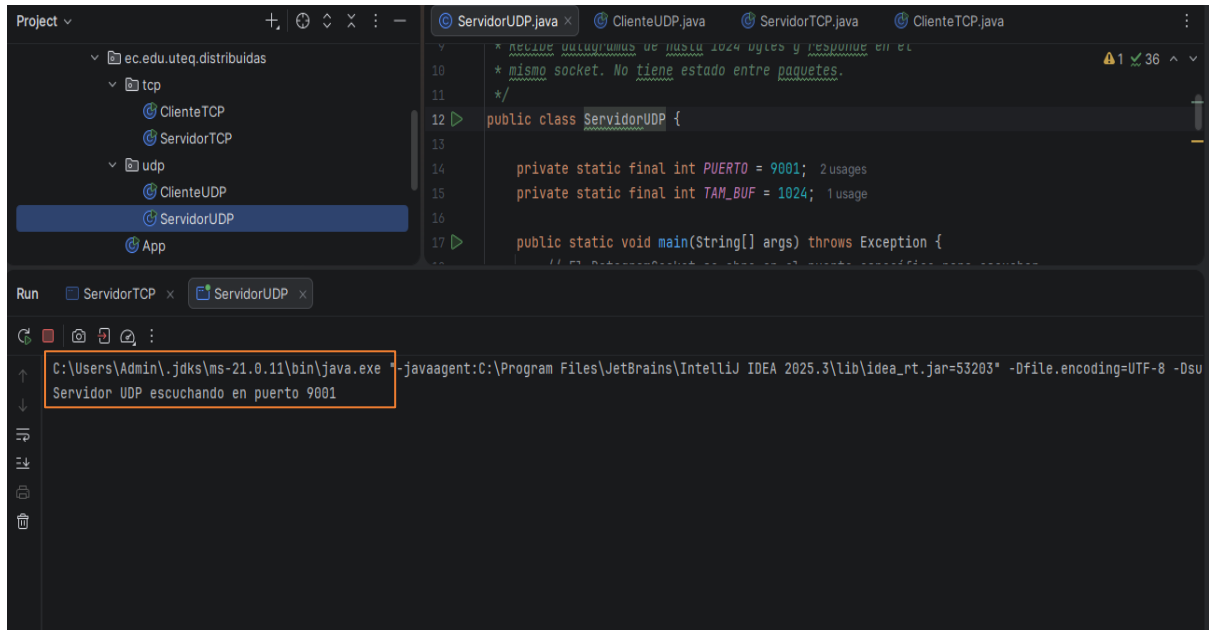


Ilustración 2 ClienteTCP

Sockets UDP

Se implementaron dos clases: un servidor que escucha en el puerto 9001, recibe mensajes y responde automáticamente con una confirmación que incluye un mensaje y la hora de recepción. El cliente envía mensajes al servidor y espera respuesta y avisa al usuario si no llega ninguna en el tiempo establecido.



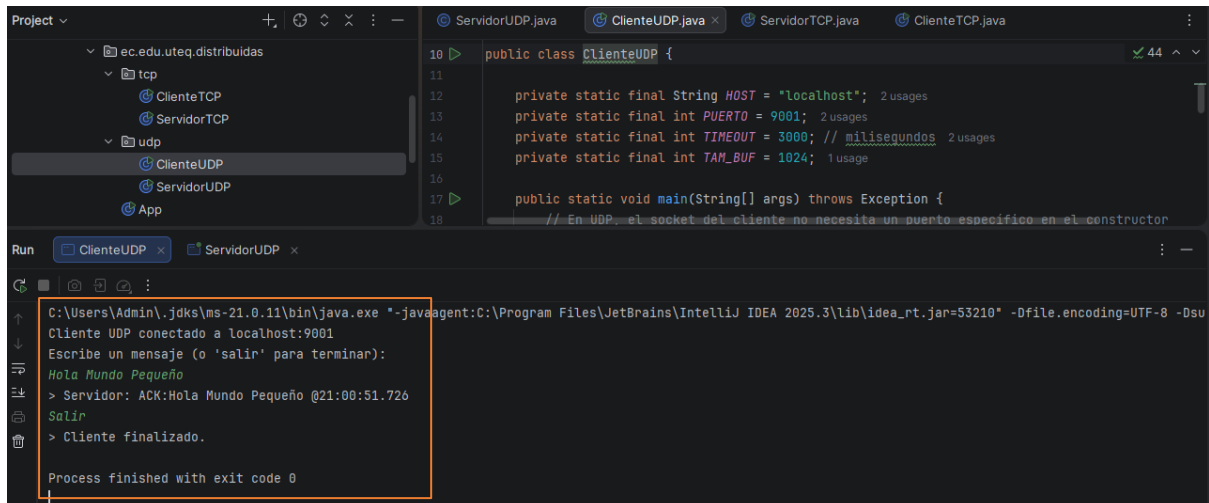
The screenshot shows the IntelliJ IDEA interface. The Project view on the left shows a directory structure with 'ec.edu.uteq.distribuidas' containing 'tcp' and 'udp' subdirectories. The 'udp' directory contains 'ClienteUDP', 'ServidorUDP', and 'App'. The 'ServidorUDP.java' file is open in the editor, showing the following code:

```
10 * recibe paquetes de hasta 1024 bytes y responde en el
11 * mismo socket. No tiene estado entre paquetes.
12 */
13
14 public class ServidorUDP {
15
16     private static final int PUERTO = 9001; 2 usages
17     private static final int TAM_BUF = 1024; 1 usage
18
19     public static void main(String[] args) throws Exception {
20         // En UDP, el socket no necesita un puerto específico en el constructor
21     }
22 }
```

The Run console at the bottom shows the command used to run the application:

```
C:\Users\Admin\jdk\ms-21.0.11\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3\lib\idea_rt.jar=53203" -Dfile.encoding=UTF-8 -Dsu
Servidor UDP escuchando en puerto 9001
```

Ilustración 3 ServidorUDP



The screenshot shows the IntelliJ IDEA interface. The Project view on the left shows the same directory structure as the previous screenshot. The 'ClienteUDP.java' file is open in the editor, showing the following code:

```
10 public class ClienteUDP {
11
12     private static final String HOST = "localhost"; 2 usages
13     private static final int PUERTO = 9001; 2 usages
14     private static final int TIMEOUT = 3000; // milisegundos 2 usages
15     private static final int TAM_BUF = 1024; 1 usage
16
17     public static void main(String[] args) throws Exception {
18         // En UDP, el socket del cliente no necesita un puerto específico en el constructor
19     }
20 }
```

The Run console at the bottom shows the command used to run the application:

```
C:\Users\Admin\jdk\ms-21.0.11\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2025.3\lib\idea_rt.jar=53210" -Dfile.encoding=UTF-8 -Dsu
Cliente UDP conectado a localhost:9001
Escribe un mensaje (o 'salir' para terminar):
Hola Mundo Pequeño
> Servidor: ACK:Hola Mundo Pequeño @21:00:51.726
Salir
> Cliente finalizado.
Process finished with exit code 0
```

Ilustración 4 ClienteUDP

Conclusión

Mediante la implementación de este sistema de mensajería distribuida se pudo comprobar en la práctica las diferencias entre TCP y UDP. El protocolo TCP es mejor cuando se necesita que los datos lleguen de forma segura y en orden, como en sistemas de mensajería o transferencia de archivos, a diferencia del protocolo UDP, en el cual la velocidad es lo más importante y puede haber pérdida de los datos.

Referencias bibliograficas

- [1] G. Coulouris, J. Dollimore, T. Kindberg, and G. Blair, "DISTRIBUTED SYSTEMS," p. 1067, 2011, Accessed: May 11, 2026. [Online]. Available: https://ftp.utcluj.ro/pub/users/civan/CPD/3.RESURSE/6.Book_2012_Distributed%20systems%20_Coulouris.pdf

Anexos

Repositorios:

<https://github.com/Ernesto835/Aplicaciones-distribuidas.git>

<https://github.com/CristhianP03/guia01-socketsv01.git>

https://github.com/Robimson/APP_distribuidas.git